

Guión de presentación

TP3 — Perceptrones

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Persona	Bloque	Slides	Tiempo
P1	Intro + Decisiones de diseño + Validación	1–14	5:30
P2	Ej 1 — Knowledge Distillation (<i>Tomás</i>)	15–21	5:30
P3	Ej 2 — Sweeps, curvas, final_eval	22–33	5:30
P4	Ej 3 + Alineación con la clase + cierre	34–47	5:30

Total: 22 minutos (más Q&A).

Foco: resultados, plots, gráficos, decisiones de diseño y por qué nos dio cada cosa.

Convenciones:

- **0:30** marca dónde debería estar el orador a los 30 segundos del bloque.
- **[Slide N]** indica cuándo cambiar de diapositiva.
- / pausa breve. // pausa más larga (cambio de tema).
- *cursivo rojo* = referencia al enunciado / apunte / clase.
- **texto en azul** = identificador del repo (mínimo).

Índice

1. Persona 1 — Intro + Decisiones de diseño + Validación

Foco: situar al jurado, fijar las cinco decisiones de diseño que aplican transversalmente al TP, y mostrar la validación con AND y XOR. La sección tiene muchas slides — cada una se queda 20–30 segundos.

Volumen objetivo: ~ 800 palabras a 145 wpm.

Apertura — 0:00 – 0:20

[Slide 1, portada]

Buenas. Trabajo Práctico tres, Perceptrones. Somos cuatro y nos vamos a turnar en orden de los ejercicios. Yo arranco con la introducción y las decisiones de diseño que afectan a todos los ejercicios, y termino con la validación del Ej 0.

Decisiones de diseño — overview 0:20 – 0:50

[Slide 2, divider Decisiones de diseño] [Slide 3, cinco decisiones]

Antes de mostrar los resultados, hay cinco elementos del MLP que tuvimos que fijar y que aplican a todo el TP. // *Función de activación*: qué transformación no-lineal usamos en cada capa. *Inicialización de pesos*: con qué valores arrancamos. *Optimizador*: cómo movemos los pesos. *Learning rate*: de qué tamaño es ese paso. Y *función de costo*: qué error estamos minimizando. / Las repaso en una slide cada una.

1. Función de activación 0:50 – 1:25

[Slide 4, activación]

Caso por caso. // En el AND del Ej 0 usamos **escalón**, porque la salida es discreta y es un perceptrón clásico de una capa, sin backprop. / En el XOR, **tanh**, que es bipolar y derivable. / En el Ej 1, **sigmoide**, porque el target es una probabilidad acotada en cero-uno. / Y en Ej 2 y Ej 3, las capas ocultas con **ReLU** y la salida con **softmax**. / ReLU porque el gradiente es fuerte y constante en zona positiva, no satura como tanh o sigmoide; softmax porque para multiclase necesitamos una distribución de probabilidad sobre las diez clases. *La clase descarta escalón (no derivable) y lineal (colapsa el MLP)*; nosotros también.

2. Inicialización de pesos 1:25 – 2:00

[Slide 5, inicialización]

El apunte dice: pequeños, no cero, no grandes. / ¿Por qué no cero? / Por simetría: si todas las neuronas arrancan iguales, el gradiente las mueve igual y nunca se diferencian. / ¿Por qué pequeños? / Porque pesos grandes saturan las activaciones, el gradiente queda casi en cero y la red no aprende. // Usamos **He** para capas con ReLU — $\mathcal{N}(0, \sqrt{2/\text{fan_in}})$ que compensa que ReLU descarta la mitad del input. / **Xavier** para tanh, sigmoide y softmax. / Y uniforme para el perceptrón simple del Ej 1.

3. Optimizador 2:00 – 2:30

[Slide 6, optimizador]

Comparamos los tres optimizadores que pide la materia. // **SGD vanilla**, la versión clásica del libro: w menos eta por gradiente. Funciona pero es lento. / **Momentum**, que acumula velocidad y atraviesa valles sin oscilar. / Y **Adam**, que combina momentum con learning rate adaptativo por parámetro. *Adam y Momentum la clase los desarrolla en la siguiente, pero ya nos pide usarlos.* // Resultado del sweep en Ej 2: Adam ganó con val accuracy 96.74 %, Momentum quedó muy cerca, y SGD no convergió en 50 epochs. / Elegimos Adam.

4. Learning rate 2:30 – 3:00

[Slide 7, learning rate]

El learning rate controla el tamaño del paso. / Demasiado grande: la red oscila o diverge. / Demasiado chico: no converge en tiempo razonable. // Hicimos dos sweeps, uno coarse en Fase 1 con cinco valores entre uno por diez a la menos cuatro y uno por diez a la menos dos. Ahí ganó **cero coma cero cero uno**. / Después en Fase 2, validación con K-fold cinco, agregamos los extremos: cero coma uno y diez. / *Diez* dejó la red en accuracy random, doce por ciento. *Cero coma uno* también divergió. Cero coma cero cero uno se confirmó como ganador.

5. Función de costo 3:00 – 3:30

[Slide 8, costo]

La clase arranca con MSE. Cambiar la función de costo cambia la derivada del backprop. / Usamos **MSE** en el Ej 0 y en el Ej 1, porque la salida es continua y queremos que sea cercana al target. / En Ej 2 y Ej 3 usamos **cross-entropy**, que es lo estándar para clasificación multiclase y penaliza fuerte la confianza en la clase incorrecta. // Combinada con softmax tiene una propiedad linda: el gradiente termina siendo simplemente la salida menos el target, sin términos numéricamente delicados.

Validación AND con escalón 3:30 – 4:00

[Slide 9, divider Bloque 1] [Slide 10, qué pide TP] [Slide 11, AND]

(Aprovecho para mostrar las dos slides anteriores rápido y entrar en validación.) // El primer caso: AND lógico con perceptrón escalón. / *Representación bipolar como pide el apunte*: las entradas en menos uno y más uno, salida esperada menos-uno menos-uno menos-uno más-uno. / Resultado: el perceptrón clasifica las cuatro muestras con error cero en pocas epochs.

Validación XOR 4:00 – 5:00

[Slide 12, XOR convergencia]

Y el caso clásico: XOR. // El XOR no es separable linealmente. Un perceptrón simple no puede resolverlo, necesita capa oculta no lineal. / Probamos dos arquitecturas: la mínima dos-dos-uno, y dos-tres-dos-uno con capa oculta extra. / En el plot, ambas convergen, eje y en escala logarítmica de loss. La de dos-dos-uno tarda más al principio pero termina en el mismo orden.

[Slide 13, qué nos dice]

// ¿Qué nos dice este resultado? / *El resultado clásico de Minsky-Papert y después Rumelhart-Hinton*: el perceptrón simple no resuelve XOR pero un MLP con una capa oculta sí. / Y para nosotros confirma que la pipeline está bien: forward pass, backpropagation, derivadas de tanh y MSE, todo correcto. La misma pipeline se reutiliza en Ej 2 y Ej 3 cambiando solo activación de salida y loss.

Cierre y transición 5:00 – 5:30**[Slide 14, transición]**

Para cerrar mi parte. // Las cinco decisiones de diseño están fijadas y la validación es positiva. / Sigue el Ej 1, que es detección de fraude por knowledge distillation. *Lo implementó Tomás antes de unificar el resto del TP en el engine genérico*, así que vive como scripts independientes. Le paso a él.

Wordcount estimado P1: $\approx$ 810 palabras. A 145 wpm $\rightarrow$ 5 : 35. Margen para pausas.

Si llegás corto: expandir el comentario sobre simetría en init de pesos.

Si llegás largo: agrupar las slides 4–8 en menos texto y entrar más rápido en la validación.

2. Persona 2 — Ej 1 — Knowledge Distillation

Foco: setup distillation, MSE lineal vs no-lineal, threshold default engañoso, sweep, ROC + PR, recomendación de umbral. Slides menos densas que P1: hay tiempo de explicar cada plot con calma.

Volumen objetivo: $\sim$ 800 palabras.

Apertura — 0:00 – 0:30**[Slide 15, divider Bloque 2] [Slide 16, problema]**

Yo me ocupo del Ej 1, que lo implementé hace un mes con scripts independientes, antes de unificar el resto en el engine. // El problema en una frase: tenemos un BigModel pre-entrenado que clasifica fraude, y nuestra tarea es entrenar un perceptrón simple — un TinyModel — que replique su salida. Eso se llama *knowledge distillation*.

// La salida del BigModel es continua, una probabilidad de fraude entre cero y uno. Los inputs son nueve features de la transacción. / Y hay una columna adicional con la etiqueta binaria de fraude real, que *el enunciado prohíbe usar para entrenar*: la usamos solo para evaluar después.

Lineal vs no-lineal: el plot del MSE 0:30 – 1:50**[Slide 17, MSE comparison]**

Probamos las dos versiones. // El perceptrón *lineal* es Adaline puro. La salida es producto punto entre pesos y entrada, sin activación. / El *no lineal* aplica una sigmoide encima del producto punto, y la regla delta gana un factor extra: la derivada de la sigmoide, que es O por uno menos O .

// Resultado: MSE del lineal, cero coma cero dos seis seis. / MSE del no lineal, cero coma cero uno uno cero. / **Reducción del cincuenta y nueve por ciento.**

// ¿Por qué gana la sigmoide? / El target del distillation está acotado entre cero y uno: es una probabilidad. / La sigmoide *naturalmente* mapea su output a ese rango. El lineal puede producir valores fuera, y el MSE penaliza esa salida fuera de dominio. No es magia: es coherencia entre la activación final y el problema.

El threshold default es engañoso 1:50 – 2:50**[Slide 18, threshold default]**

Acá viene la parte interesante. // Si tomamos el modelo no lineal con threshold cero coma cinco, las métricas operativas son malas. / Precision treinta y tres por ciento, recall cien por ciento. Clasifica casi todo como fraude. / Eso a primera vista parece un modelo terrible, pero el modelo no es el problema: *el threshold no está calibrado*.

// ¿Por qué? / Porque las salidas sigmoide se concentran en la parte alta. La mediana de los scores de fraude está en cero coma noventa y nueve. / Y los no-fraudes no se van a cero coma cero

cinco como uno esperaría: están en cero coma cuatro, cero coma seis. / El modelo aprendió un buen *ranking*, pero las probabilidades absolutas no están calibradas. Threshold bajo se traduce en sobre-detección.

Threshold sweep 2:50 – 3:50

[Slide 19, threshold sweep]

La solución es barrer el threshold. // En el plot ven precision en azul, recall en verde, F1 en negro, todo en función del threshold de cero a uno.

// A la izquierda, threshold cero: precision igual a la prevalencia de fraude, recall uno. / A la derecha, threshold uno: precision uno, recall casi cero. / El óptimo por F1 está en **threshold cero coma ochenta y nueve**: precision ochenta y nueve, recall ochenta y seis, F1 ochenta y siete, accuracy noventa y siete por ciento.

ROC y PR 3:50 – 4:30

[Slide 20, ROC + PR]

Las dos curvas que faltan. // A la izquierda, ROC: tasa de verdaderos positivos contra falsos positivos. AUC del modelo no lineal: **cero coma noventa y nueve veintiuno**. / A la derecha, precision-recall, que es más informativa para clases desbalanceadas como fraude — la prevalencia es del orden del doce por ciento. / La capacidad de discriminación global es excelente; lo único es decidir dónde cortar.

Recomendación de umbral 4:30 – 5:30

[Slide 21, recomendación]

Para cerrar. // ¿Qué umbral recomendamos al cliente? Depende del costo asimétrico. // **Threshold cero coma ochenta y nueve** si la auditoría manual es barata — maximiza F1, captura el ochenta y seis por ciento de los fraudes. // **Threshold mayor o igual a cero coma noventa y cinco** si los falsos positivos son caros — precision pasa de noventa y cinco por ciento pero recall baja a setenta y cinco. // El modelo entrega scores; la política de threshold la define el dueño del producto. Le paso al siguiente.

Wordcount estimado P2: ≈ 800 palabras. A 145 wpm $\rightarrow 5 : 30$.

Si llegás corto: expandir cómo se calculó el AUC sin sklearn.

Si llegás largo: no detallar la opción del threshold 0.95.

3. Persona 3 — Ej 2 — Clasificación de dígitos

Foco: workflow Fase 1+2, plots de los sweeps separados, learning curves, matriz, y final_eval con el cachetazo del distribution shift como gancho para Persona 4. Bloque denso (12 slides), cada slide queda en ~ 25 segundos.

Volumen objetivo: ~ 800 palabras.

Problema y datos 0:00 – 0:30

[Slide 22, divider Bloque 3] [Slide 23, problema]

Yo cuento el Ej 2: clasificación de dígitos manuscritos del cero al nueve. // Cada muestra es una imagen de veintiocho por veintiocho aplanada en *un vector de setecientos ochenta y cuatro floats entre cero y uno*. Train de doce mil muestras, test de dos mil quinientas. / Y, como dijo P1, el test

se evalúa una sola vez al final. // La arquitectura final, después de los sweeps, es setecientos ochenta y cuatro - cien - cincuenta - diez. ReLU en hiddens, softmax en salida.

Mini-batch SGD 0:30 – 1:00

[Slide 24, mini-batch SGD]

Detalle de implementación rápido. // En Ej 2 y Ej 3 usamos **mini-batch SGD**, no online estricto como en Ej 1. / El motivo: en datasets grandes, online es muy lento porque no aprovecha la matmul vectorizada de NumPy, y batch puro es muy costoso por update. / Mini-batch es el intermedio: gradiente más estable que online y más rápido que batch. / El tamaño del batch lo determinamos en un sweep de la Fase 1.

Workflow Fase 1 + Fase 2 1:00 – 1:50

[Slide 25, workflow]

Para no probar hiperparámetros al azar, dividimos la búsqueda en dos fases. // *Fase uno, exploratoria*, con K-folds igual a uno — un solo train-val split. / Sweeps secuenciales: arquitectura, después optimizador, después learning rate, después batch size. / Cada sub-fase usa el ganador del anterior. // *Fase dos, validación*, con K-fold igual a cinco. / Tomamos el config ganador de Fase uno y variamos un solo HP a la vez. / Y al final, un único final_eval contra el test.

Sweeps de Fase 2 1:50 – 3:00

[Slide 26, sweep LR]

Empiezo por los plots de Fase 2. // *Learning rate*. / El único sweep con rangos catastróficos: lr=10 deja la red en accuracy random, lr=0.1 también diverge. lr=0.001 confirma la elección de Fase 1.

[Slide 27, sweep arquitectura]

// *Arquitectura*. / Probamos cuatro variantes alrededor del ganador. La versión *wider* con doscientas-cien gana en val por cero coma diecisiete puntos, pero tarda *tres veces y media* más en entrenar. / Empate técnico, mantenemos la base por parsimonia.

[Slide 28, sweep optimizador]

// *Optimizador*. / Adam y Momentum empatan dentro del desvío. SGD claramente atrás — no convergió en cincuenta epochs sin aceleración. Confirma a Adam.

[Slide 29, lo que confirmó Fase 2]

// Take-away transversal: / Fase 2 *valida más que descubre*. Solo el sweep de LR muestra rangos catastróficos. Los demás quedan en empate técnico. / La metodología coarse-to-fine de Fase 1 ya hizo el grueso del trabajo.

Curvas y matriz del base 3:00 – 4:00

[Slide 30, curvas base]

Ahora el modelo base con K-fold cinco. // Las curvas de aprendizaje. / Train y validación bajan juntos, convergencia en cinco epochs. Sin overfitting visible.

[Slide 31, matriz base]

// La matriz de confusión sobre val. / Diagonal limpia para nueve de las diez clases. / La excepción — y esto es importante — es la **clase ocho**: precision cero, recall cero. La red nunca predice ocho. / Ya nos dio una pista de que algo raro pasaba con la distribución. Quédense con esto.

Final eval — el cachetazo 4:00 – 5:30

[Slide 32, cachetazo]

Y entonces hicimos el final eval. / Entrenamos con todo `digits.csv` y evaluamos una sola vez contra `digits_test.csv`. // Resultado.

// Val K-fold cinco: noventa y seis coma veintidós por ciento. / Test sobre digits_test: **ochenta y seis coma treinta por ciento.** // **Drop de diez puntos porcentuales.**

[Slide 33, matriz final test]

/ La matriz de confusión sobre el test set confirma que las confusiones se reparten por todo el grid: no es una clase específica que falla. // El patrón es *distribution shift*. / **El modelo está bien. Los datos no se parecen.** // Si eso es cierto, lo que necesitamos no es un modelo más grande, sino más datos que cubran la distribución del test. *Y eso es lo que el Ej 3 nos provee.* Le paso al siguiente.

Wordcount estimado P3: ≈ 810 palabras. A 145 wpm $\rightarrow 5 : 35$.

Si llegás corto: expandir el comentario sobre la clase 8 colapsada (hipótesis: sub-representación en train).

Si llegás largo: agrupar los tres sweeps Fase 2 en una sola descripción global.

4. Persona 4 — Ej 3 + Alineación con la clase + Conclusiones

Foco: hipótesis y plan, +10pp por más datos, Pack C explainer, tabla de resultados, comparison plot, modelo ganador, análisis qué movió la aguja, hipótesis del techo, alineación con la clase, conclusiones.

Bloque más denso (14 slides), ~ 22 segundos por slide.

Volumen objetivo: ~ 810 palabras.

Plan del Ej 3 0:00 – 0:30

[Slide 34, divider Bloque 4] [Slide 35, hipótesis y plan]

Yo cierro con el Ej 3 y las conclusiones. // La hipótesis que dejó la persona anterior es que el problema es *distribution shift*, no falta de capacidad. Si eso es cierto, lo primero que hay que probar es *más datos*, no un modelo más grande. *Y el enunciado del Ej 3 nos provee `more_digits.csv`*, con quince mil setecientas muestras adicionales.

// Plan secuencial: primero, más datos sin tocar el modelo. / Segundo, si no alcanza, activar Pack C: la regularización del enunciado — L2, dropout, augmentation gaussiana, lr scheduling. / Tercero, si nada llega, escalar capacidad.

El primer dominó 0:30 – 1:20

[Slide 36, dominó]

El primer movimiento fue agregar `more_digits.csv`. / **Cero cambios al modelo**, solo concatenamos los dos CSVs. // Resultado.

// Val K-fold cinco subió de noventa y seis coma veintidós a noventa y siete coma cero seis. / Pero lo importante es el test: **de ochenta y seis coma treinta a noventa y seis coma treinta y seis.** // **Diez puntos porcentuales en test sin tocar el modelo.** / Y la clase ocho que estaba colapsada ahora se predice correctamente. Macro F1 de cero coma ochenta y cinco a cero coma noventa y cinco.

¿Qué es Pack C? 1:20 – 1:50

[Slide 37, Pack C explainer]

Una aclaración antes de los resultados de Pack C. // “Pack C” es nomenclatura interna del proyecto, no del enunciado. / Agrupamos las capacidades del engine en tres niveles. / Pack A es el perceptrón clásico — escalón, regla delta básica, que usamos en Ej 0 AND y Ej 1. / Pack B es lo

estándar académico: activaciones, optimizadores SGD/Momentum/Adam, He y Xavier, mini-batch, K-fold. Es lo que usa todo Ej 2 y el baseline de Ej 3. // Y Pack C son los módulos opcionales de regularización: L2, dropout, lr scheduling, augmentation gaussiana. / Por qué los separamos: para mantenerlos enchufables y desactivados por default. Si Ej 3 con más datos hubiera llegado al noventa y ocho, Pack C no se activaba.

Tabla Pack C 1:50 – 2:50

[Slide 38, tabla Pack C]

Pero faltaba uno coma sesenta y cuatro puntos para llegar al target. Así que probamos Pack C. // La tabla muestra todas las configs. / Vamos por la columna de test.

L2 con weight decay uno por diez a la menos cuatro: noventa y seis coma cincuenta y seis. Cero coma veinte sobre el baseline. / **Dropout solo**: noventa y seis coma veintiocho. Peor. / **L2 más dropout**: noventa y seis coma cero cuatro. Aún peor. // La fila resaltada en verde, **L2 más augmentation gaussiana sigma cero coma cero cinco**: **noventa y seis coma ochenta y ocho**. / Ese es el ganador. // Las otras combinaciones más abajo no superan a esta.

Comparación visual y modelo ganador 2:50 – 3:50

[Slide 39, comparison Ej2 vs Ej3]

Comparación entre Ej 2 y Ej 3. / La línea roja es el target del noventa y ocho por ciento. El salto principal viene del baseline con extra data, no de la regularización fina.

[Slide 40, curvas ganador]

// Curvas de aprendizaje del ganador. / Convergencia un poco más lenta que el Ej 2 base por la regularización. Sin overfitting.

[Slide 41, matriz ganador]

// Matriz de confusión sobre test. / La clase ocho que estaba colapsada ahora se predice. Errores residuales repartidos — ya no hay agujero específico, solo limitaciones globales.

Análisis: qué movió la aguja 3:50 – 4:30

[Slide 42, qué ayudó qué no]

El análisis honesto. // **Lo que ayudó**: más datos diez puntos, L2 cero coma veinte, augmentation cero coma treinta y dos puntos adicionales sobre L2. / **Lo que no ayudó**: dropout gana en val pero pierde en test — reduce varianza al fold, no al shift. / La arquitectura wider memoriza el train, no generaliza. Eso *descarta que el problema sea capacidad*. / Y combinar todo a la vez fue peor que combinar L2 más augmentation solas: la regularización compite consigo misma cuando se exagera.

Por qué no llegamos al 98 4:30 – 4:50

[Slide 43, hipótesis del techo]

¿Por qué no llegamos al noventa y ocho? Mejor noventa y seis coma ochenta y ocho, faltan uno coma doce. // Hipótesis principal: **augmentation insuficiente**. Gaussian noise es *isotrópico* — ruido independiente por píxel. El shift parece ser de *estilo de escritura*, que pide augmentation *geométrica*: rotación, traslación. No la implementamos. / Es el follow-up natural.

Alineación con la clase 4:50 – 5:15

[Slide 44, lo que seguimos del apunte]

Antes de cerrar, una mención rápida. // **Lo que seguimos del apunte**: backprop con regla de la cadena y delta retropropagada, bias en cada capa, forma matricial sin loops por neurona, init pequeña con He y Xavier, comparación SGD/Momentum/Adam, sweep de LR en varios órdenes con extremos, error en función de épocas, online en Ej 1 y mini-batch en Ej 2 y 3.

[Slide 45, decisiones distintas]

Decisiones distintas: no barrimos online vs batch vs mini-batch como hiperparámetro — mini-batch domina por costo. / No barrimos funciones de activación — la clase descarta escalón y lineal, softmax es obligatorio en multiclase, ReLU es estándar. / Convergencia por early stopping en validación, no ε absoluto. / Y la estructura del TP cambió respecto a la clase: el enunciado actual difiere.

Conclusiones 5:15 – 5:30

[Slide 46, conclusiones]

Cierro con tres ideas. // Una: una buena pipeline gana más que tunear hiperparámetros. / Dos: los empates técnicos son más comunes que los wins claros, hay que elegir por parsimonia. / Tres: “mejor en val” no es “mejor en test” cuando hay shift. // Cumplimos todo lo del enunciado excepto el target del noventa y ocho. [Slide 47, gracias] Gracias. Preguntas.

Wordcount estimado P4: ≈ 820 palabras. A 145 wpm $\rightarrow 5 : 39$. Apretado, cuidar el ritmo.

Si llegás corto: expandir la hipótesis tres del techo (capacidad).

Si llegás largo: comprimir slides 44–45 a una sola frase: “seguimos todo el apunte excepto cuatro decisiones que justifiqué en el informe”.

Apéndice — preguntas anticipadas

¿Por qué no usaron sklearn o PyTorch?

El enunciado lo prohíbe explícitamente. La consigna pide perceptrones desde cero con NumPy.

¿Por qué dos fases en el Ej 2?

Para no caer en sweep aleatorio. Fase 1 con `k_folds=1` explora rápido y reduce el espacio. Fase 2 con `K=5` valida con K-fold completo.

¿Qué es Adam? ¿No se ve en la clase siguiente? (P1 o P4)

Sí, la clase de backprop solo lo menciona; lo desarrollan después. Implementamos las fórmulas siguiendo el paper original (Kingma & Ba 2014). En esencia: combina momentum (promedio del gradiente) con RMSProp (promedio del gradiente al cuadrado, para LR adaptativo por parámetro). Los hiperparámetros default son $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$.

¿Por qué He para ReLU y Xavier para tanh? (P1)

Porque la varianza de las activaciones depende de la activación. ReLU descarta la mitad del input (todo lo negativo se va a cero), entonces para mantener varianza balanceada por capa hay que multiplicar por dos. Xavier asume activaciones simétricas alrededor de cero (tanh, sigmoide).

¿Por qué eligieron threshold 0.89? (P2)

Maximiza F1 en el sweep. Si el cliente prefiere precisión sobre recall, `threshold  $\geq 0,95$` . Es decisión de negocio sobre el costo asimétrico FN/FP, no del modelo.

¿Por qué dropout no funcionó en el Ej 3? (P4)

Dropout reduce varianza dentro de la distribución del train. Acá el problema no es varianza dentro del train: es shift entre train y test. Dropout puede ayudar en val K-fold, donde las clases son las mismas que en train, pero no transfiere a un test que viene de una distribución

parcialmente distinta.

¿Por qué no llegan al 98 % del Ej 3? (versión hostil — P4)

Identificamos que el techo viene de la calidad del augmentation, no de la capacidad del modelo. Probamos las cuatro técnicas Pack C que sugiere el enunciado y siete combinaciones distintas. Lo que faltaba implementar es augmentation por transformaciones *geométricas* — rotación pequeña, traslación. Es follow-up explícito.

¿Por qué el Ej 1 está aparte del engine? (P2)

Lo implementé hace un mes con scripts independientes, antes de unificar la codebase. Funcionalmente correcto. Mantener la implementación separada preserva la trazabilidad del repo.

Fin del guion. Slides en `docs/slides_presentacion.pdf`.